

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

JAVASCRIPT FUNCTIONS, CLASSES AND OBJECT CREATION (CO-1)

1. TITLE

Differentiate Between JavaScript Functions and Classes, and Explain How a Class Can Be Used to Create Multiple Objects with Common Properties and Methods.

2. AIM

To study the difference between JavaScript functions and classes, and to develop a program in which a single class is used as a blueprint to create multiple objects that share the same properties and methods.

3. OBJECTIVE

- To understand the role of an ordinary function and a constructor function in JavaScript.
- To understand the ES6 class syntax: class, constructor(), instance methods, and static members.
- To differentiate between functions and classes with respect to syntax, hoisting, invocation, and strict mode.
- To create multiple objects from one class using the new keyword.
- To understand that properties are stored separately in every object while methods are shared through the prototype.
- To verify the behaviour by executing the program in Node.js.

4. THEORY

1. Functions in JavaScript

A function is a reusable block of code that performs a specific task. It is declared with the function keyword, may accept parameters, and may return a value. In JavaScript, functions are first-class objects: they can be stored in variables, passed as arguments, and returned from other functions. A function declaration is hoisted, so it can be called even before the line in which it is written.

Before ES6, a function was also used to create objects. Such a function is called a **constructor function**. When it is called with the new keyword, JavaScript creates an empty object, binds it to this, executes the function body, and returns the object. Shared methods have to be attached manually through `FunctionName.prototype`.

2. Classes in JavaScript

A class, introduced in ES6 (ECMAScript 2015), is a template (blueprint) for creating objects. It is mainly syntactic sugar over the existing prototype-based inheritance, but it provides a cleaner and more readable syntax. A class body can contain a `constructor()`, instance methods, static members, and getters/setters, and a class can be extended using the `extends` keyword. Internally, `typeof` a class is still `'function'`.

3. Difference Between Functions and Classes

Feature	Function (constructor function)	Class
Syntax	<code>function Name(p) { this.x = p; }</code>	<code>class Name { constructor(p) { this.x = p; } }</code>
Purpose	General purpose: performs a task and may return a value; can also act as an object constructor.	Designed specially as a blueprint for creating objects.
Hoisting	Function declarations are hoisted and can be called before their definition.	Not hoisted for use; accessing before declaration gives <code>ReferenceError</code> (temporal dead zone).
Calling without new	Allowed. Runs as a normal function; no error is raised and <code>this</code> may point to the global object.	Not allowed. Throws a <code>TypeError</code> .
Methods	Added manually: <code>Name.prototype.m = function(){} </code>	Written inside the class body and automatically stored on the prototype.
Strict mode	Non-strict by default.	Class body is always executed in strict mode.
Static members	Assigned as properties: <code>Name.count = 0</code>	Declared with the <code>static</code> keyword.
Inheritance	Manual prototype chaining; complex.	Simple, using <code>extends</code> and <code>super</code> .
Method enumeration	Methods added to the prototype are enumerable.	Class methods are non-enumerable.

4. Objects, Properties and Methods

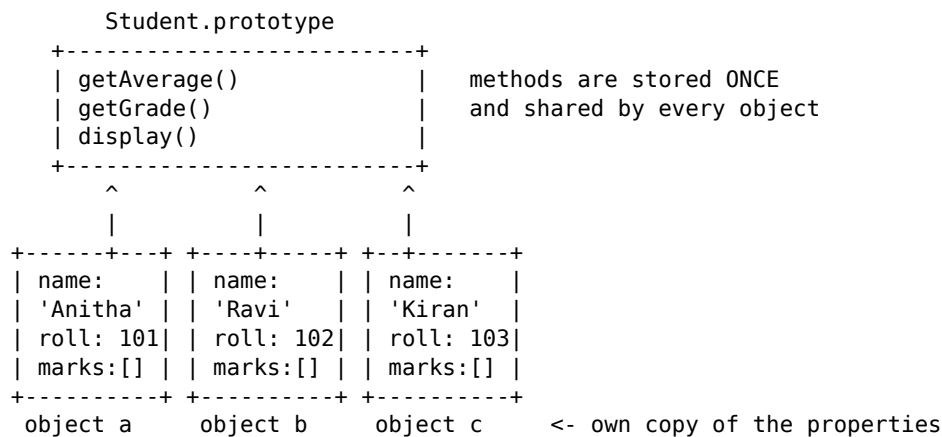
An object is a collection of related data and behaviour. The data stored in an object are its **properties** (for example name, roll, marks) and the functions that work on this data are its **methods** (for example `getAverage()`, `getGrade()`). An object created from a class is called an **instance** of that class.

5. Creating Multiple Objects from One Class

When `new Student(...)` is executed, JavaScript performs four steps:

- It creates a new empty object.
- It links the prototype of the object to `Student.prototype`.
- It executes `constructor()` with `this` pointing to the new object, so the property values are stored in that object.
- It returns the newly created object.

Every object therefore has its own copy of the properties, and changing one object never affects another. The methods, however, are defined only once on `Student.prototype`, and every object reaches them through the prototype chain. Thus all objects have **common properties** (same property names) and **common methods** (same code), while the values of the properties are different. This saves memory and avoids duplication of code.



6. Static Members

A static property or method belongs to the class itself and not to its objects. It is accessed using the class name (for example `Student.getCount()`) and is useful for data that is common to the whole class, such as the number of objects created.

5. ALGORITHM / PROCEDURE

1. Start.
2. Create a file named `classes.js`.
3. Define an ordinary function `add()` that returns the sum of two numbers.
4. Define a constructor function `StudentFn()` and attach `getAverage()` to `StudentFn.prototype`.
5. Define a class `Student` with a static property `count`, a `constructor()` that assigns `name`, `roll`, and `marks`, the instance methods `getAverage()`, `getGrade()`, and `display()`, and the static method `getCount()`.
6. Define `demo1()` to call the ordinary function and to create objects using the constructor function.
7. Define `demo2()` to create three objects from the `Student` class and display their details.

8. Define demo3() to prove that the methods are shared through the prototype while the properties are separate for every object.
9. Define demo4() to show the behavioural differences: calling without new, and hoisting.
10. Define demo5() to show that objects are independent and that static members are not available on objects.
11. Read the demo number from the command line and call the selected demo (or all demos when no number is given).
12. Execute the file using node classes.js and observe the output.
13. Stop.

6. PROGRAM

classes.js

```
/**
 * classes.js
 * Experiment 5 - JavaScript Functions vs Classes
 * Usage: node classes.js      (runs every demo)
 *        node classes.js 3    (runs only demo number 3)
 */

// ----- 1. Ordinary function and constructor function -----
// Ordinary function: performs an action
function add(a, b) {
  return a + b;
}

// Constructor function (pre-ES6 way)
function StudentFn(name, roll, marks) {
  this.name = name;
  this.roll = roll;
  this.marks = marks;
}
// Method shared via prototype
StudentFn.prototype.getAverage = function () {
  return this.marks.reduce((sum, m) => sum + m, 0) / this.marks.length;
};

// ----- 2. Class (ES6) : blueprint for many objects -----
class Student {
  // Static property: belongs to the class
  static count = 0;

  // Runs automatically for every new object
  constructor(name, roll, marks) {
    // Instance properties (separate for each object)
    this.name = name;
    this.roll = roll;
    this.marks = marks;
    Student.count++;
  }

  // Instance methods: stored once on the prototype
  getAverage() {
    return this.marks.reduce((sum, m) => sum + m, 0) / this.marks.length;
  }

  getGrade() {
    const avg = this.getAverage();
    if (avg >= 90) return 'A+';
    if (avg >= 75) return 'A';
    if (avg >= 60) return 'B';
    return 'C';
  }

  display() {
    const avg = this.getAverage().toFixed(2);
    const grade = this.getGrade();
    console.log(
```

```

        `${this.roll} | ${this.name.padEnd(7)} | Avg: ${avg} | Grade: ${grade}`
    );
}

// Static method: called on the class
static getCount() {
    return Student.count;
}
}

// ----- Demo 1 : function vs constructor function -----
function demo1() {
    console.log('--- Demo 1: Function and Constructor Function ---');
    // Call the ordinary function
    console.log('add(10, 20)          =', add(10, 20));
    // Create two objects using the constructor function
    const f1 = new StudentFn('Anitha', 101, [80, 90, 70]);
    const f2 = new StudentFn('Ravi', 102, [60, 75, 85]);
    // Display an object and call its shared method
    console.log('f1                  =', f1);
    console.log('f2.getAverage()      =', f2.getAverage().toFixed(2));
    // Check the type of both
    console.log('typeof add           =', typeof add);
    console.log('typeof StudentFn     =', typeof StudentFn);
}

// ----- Demo 2 : one class, many objects -----
function demo2() {
    console.log('--- Demo 2: One Class, Many Objects ---');
    // Create three objects from the same class
    const students = [
        new Student('Anitha', 101, [88, 92, 95]),
        new Student('Ravi', 102, [65, 70, 72]),
        new Student('Kiran', 103, [78, 82, 80]),
    ];
    // Every object uses the same display() method
    students.forEach(s => s.display());
    // Read the static counter through the class
    console.log('Objects created so far:', Student.getCount());
}

// ----- Demo 3 : common methods, separate data -----
function demo3() {
    console.log('--- Demo 3: Common Methods, Separate Data ---');
    // Create two objects
    const a = new Student('Anitha', 101, [88, 92, 95]);
    const b = new Student('Ravi', 102, [65, 70, 72]);
    // Both objects use the same method from the prototype
    console.log('a.getGrade === b.getGrade      :', a.getGrade === b.getGrade);
    console.log('"a.hasOwnProperty(\'name\')      :"', a.hasOwnProperty('name'));
    console.log('"a.hasOwnProperty(\'getGrade\')      :"',
        a.hasOwnProperty('getGrade'));
    // Confirm the link between the objects and the class
    const sameProto = Object.getPrototypeOf(a) === Student.prototype;
    console.log('getPrototypeOf(a) === Student.prototype:', sameProto);
    console.log('a instanceof Student          :', a instanceof Student);
    // The property values are different for each object
    console.log('a.name / b.name                :', a.name, '/', b.name);
    console.log('a.getGrade() / b.getGrade()        :',

```

```

        a.getGrade(), '/', b.getGrade());
    }

// ----- Demo 4 : behavioural differences -----
function demo4() {
    console.log('--- Demo 4: Behavioural Differences ---');
    // A class is a special kind of function
    console.log('typeof Student          :', typeof Student);
    try {
        // Class called without new
        Student('Test', 1, [50]);
    } catch (e) {
        console.log('Class called without new      ->',
            e.name + ':', e.message);
    }
    // Function called without new
    const r = StudentFn('Test', 1, [50]);
    console.log('Function called without new      -> returned', r,
        '(no error, silent bug)');
    // Function declarations are hoisted
    console.log('Function used before declaration ->', hoisted());
    function hoisted() { return 'works, function declarations are hoisted'; }
    try {
        // Class used before declaration
        new Later();
    } catch (e) {
        console.log('Class used before declaration ->',
            e.name + ':', e.message);
    }
    class Later {}
}

// ----- Demo 5 : objects are independent -----
function demo5() {
    console.log('--- Demo 5: Objects Are Independent ---');
    const x = new Student('Anitha', 101, [90, 90, 90]);
    const y = new Student('Ravi', 102, [50, 60, 70]);
    // Change a property of x only
    x.name = 'Anitha K';
    // Add a new property to x only
    x.college = 'Sample College';
    // The change is visible only in x
    console.log('x.name / x.college          :', x.name, '/', x.college);
    console.log('y.name / y.college          :', y.name, '/', y.college);
    // Static members are not available on objects
    console.log('typeof x.getCount          :', typeof x.getCount,
        '(static: not on objects)');
    console.log('Student.getCount()          :', Student.getCount(),
        '(called on the class)');
}

// ----- Running the demos -----
// Demo functions in order (the demo number is given on the command line)
const demos = { 1: demo1, 2: demo2, 3: demo3, 4: demo4, 5: demo5 };
const choice = process.argv[2];
// Run the selected demo, or every demo when no number is given
if (choice && demos[choice]) {
    demos[choice]();
} else {

```

```
Object.values(demos).forEach(fn => { fn(); console.log(); });  
}
```

7. CODE EXPLANATION

Code	Explanation
<code>function add(a, b) { return a + b; }</code>	Ordinary function. It performs a task, returns a value, is hoisted, and is called without new.
<code>function StudentFn(name, roll, marks) { this.name = name; ... }</code>	Constructor function. Used with new to create objects; this refers to the new object.
<code>StudentFn.prototype.getAverage = function () {...}</code>	Adds a method manually to the prototype so that all StudentFn objects share one copy.
<code>class Student { ... }</code>	Declares the class, which is the blueprint for the objects. It is not hoisted.
<code>static count = 0;</code>	Static property that belongs to the class. It counts how many objects are created.
<code>constructor(name, roll, marks)</code>	Runs automatically on new and stores the given values in the properties of the new object.
<code>Student.count++</code>	Increments the static counter every time an object is created.
<code>getAverage(), getGrade(), display()</code>	Instance methods. They are stored once on Student.prototype and are available to every object.
<code>static getCount()</code>	Static method. It is called as Student.getCount() and is not available on objects.
<code>new Student('Anitha', 101, [88, 92, 95])</code>	Creates a new object (instance) of the class and calls the constructor.
<code>a.getGrade === b.getGrade</code>	true, because both objects use the same function stored on the prototype.
<code>a.hasOwnProperty('name')</code>	true, since properties are stored inside the object. For 'getGrade' it is false because methods are on the prototype.
<code>Object.getPrototypeOf(a) === Student.prototype</code>	Confirms that the object is linked to the prototype of its class.
<code>a instanceof Student</code>	Checks whether the object was created from the class.
<code>Student('Test', 1, [50])</code>	Calling a class without new throws a TypeError.
<code>new Later(); class Later {}</code>	Using a class before its declaration throws a ReferenceError; classes are not hoisted like functions.
<code>process.argv[2]</code>	Reads the demo number typed on the command line (node classes.js 3).

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the program in Node.js. The demo number is passed as a command-line argument.

Test Case TC-01

Test / Input	Run node classes.js 1.
Expected Result	The ordinary function returns 30. Objects are created from the constructor function using new, and typeof shows function for both the function and the constructor function.
Actual Result	add(10, 20) printed 30. The object f1 was displayed as StudentFn { ... }, f2.getAverage() returned 73.33, and typeof returned function for both.
Status	Pass

TC-01 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp5>node classes.js 1
--- Demo 1: Function and Constructor Function ---
add(10, 20)           = 30
f1                    = StudentFn { name: 'Anitha', roll: 101, marks: [ 80, 90, 70 ] }
f2.getAverage()       = 73.33
typeof add             = function
typeof StudentFn      = function
C:\FSWD\Exp5>_
```

Test Case TC-02

Test / Input	Run node classes.js 2.
Expected Result	Three objects are created from the Student class. Each object shows its own name, roll number, average, and grade, and the static counter shows 3.
Actual Result	Three lines were displayed: Anitha (91.67, A+), Ravi (69.00, B), and Kiran (80.00, A). The static counter showed 3.
Status	Pass

TC-02 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp5>node classes.js 2
--- Demo 2: One Class, Many Objects ---
101 | Anitha | Avg: 91.67 | Grade: A+
102 | Ravi  | Avg: 69.00 | Grade: B
103 | Kiran  | Avg: 80.00 | Grade: A
Objects created so far : 3
C:\FSWD\Exp5>_
```

Test Case TC-03

Test / Input	Run node classes.js 3.
Expected Result	Both objects share the same method, name is an own property but getGrade is not, and both objects are linked to Student.prototype, while their property values differ.
Actual Result	a.getGrade === b.getGrade returned true, hasOwnProperty('name') returned true, hasOwnProperty('getGrade') returned false, and the prototype check returned true. The grades were A+ and B.
Status	Pass

TC-03 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp5>node classes.js 3
--- Demo 3: Common Methods, Separate Data ---
a.getGrade === b.getGrade           : true
a.hasOwnProperty('name')            : true
a.hasOwnProperty('getGrade')        : false
getPrototypeOf(a) === Student.prototype : true
a instanceof Student                 : true
a.name / b.name                     : Anitha / Ravi
a.getGrade() / b.getGrade()         : A+ / B
C:\FSWD\Exp5>_
```

Test Case TC-04

Test / Input	Run node classes.js 4.
Expected Result	A class called without new throws TypeError. A function called without new gives no error. A function declaration is hoisted, but a class used before its declaration throws ReferenceError.
Actual Result	TypeError was thrown for the class. The function returned undefined without any error. The hoisted function worked, and ReferenceError was thrown for the class used before initialization.
Status	Pass

TC-04 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp5>node classes.js 4
--- Demo 4: Behavioural Differences ---
typeof Student                       : function
Class called without new              -> TypeError: Class constructor Student cannot be invoked without 'new'
Function called without new           -> returned undefined (no error, silent bug)
Function used before declaration      -> works, function declarations are hoisted
Class used before declaration         -> ReferenceError: Cannot access 'Later' before initialization
C:\FSWD\Exp5>_
```

Test Case TC-05

Test / Input	Run node classes.js 5.
Expected Result	Changing a property and adding a new property affects only that object. The static method is not available on the object but works on the class.
Actual Result	x showed Anitha K / Sample College while y showed Ravi / undefined. typeof x.getCount was undefined, and Student.getCount() returned 2.
Status	Pass

TC-05 Output:

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp5>node classes.js 5
--- Demo 5: Objects Are Independent ---
x.name / x.college      : Anitha K / Sample College
y.name / y.college      : Ravi / undefined
typeof x.getCount       : undefined (static: not on objects)
Student.getCount()      : 2 (called on the class)
C:\FSWD\Exp5>_
```

9. OUTPUT

Output 1: Complete execution of classes.js (all five demos)

```
C:\Windows\System32\cmd.exe
C:\FSWD\Exp5>node classes.js
--- Demo 1: Function and Constructor Function ---
add(10, 20)           = 30
f1                    = StudentFn { name: 'Anitha', roll: 101, marks: [ 80, 90, 70 ] }
f2.getAverage()       = 73.33
typeof add            = function
typeof StudentFn      = function

--- Demo 2: One Class, Many Objects ---
101 | Anitha | Avg: 91.67 | Grade: A+
102 | Ravi  | Avg: 69.00 | Grade: B
103 | Kiran | Avg: 80.00 | Grade: A
Objects created so far : 3

--- Demo 3: Common Methods, Separate Data ---
a.getGrade === b.getGrade      : true
a.hasOwnProperty('name')       : true
a.hasOwnProperty('getGrade')   : false
getPrototypeOf(a) === Student.prototype : true
a instanceof Student           : true
a.name / b.name                : Anitha / Ravi
a.getGrade() / b.getGrade()    : A+ / B

--- Demo 4: Behavioural Differences ---
typeof Student                  : function
Class called without new        -> TypeError: Class constructor Student cannot be invoked without 'new'
Function called without new     -> returned undefined (no error, silent bug)
Function used before declaration -> works, function declarations are hoisted
Class used before declaration   -> ReferenceError: Cannot access 'Later' before initialization

--- Demo 5: Objects Are Independent ---
x.name / x.college              : Anitha K / Sample College
y.name / y.college              : Ravi / undefined
typeof x.getCount               : undefined (static: not on objects)
Student.getCount()              : 7 (called on the class)
C:\FSWD\Exp5>
```

Figure 1: Output of node classes.js

10. RESULT

Thus, the difference between JavaScript **functions** and **classes** was studied, and a program was developed in which the class Student was used as a blueprint to create multiple objects. The output confirmed that every object keeps its own property values while all objects share the same methods through Student.prototype, that a class must be invoked with new and is not hoisted, and that static members belong to the class and not to its objects.